

IoT Security

Title: IoT Security BoT

Author: Kevin Saavedra

1. ABSTRACT

This project, IoTBot Security, implements a robust and intelligent IoT defense system using MQTT with TLS, authentication, and HMAC verification. It integrates gas, temperature, and motion sensors with actuators (smart door, HVAC, gas alarm) managed by a central hub. The core component, IoTBot, continuously monitors all traffic to detect DDoS, spoofing, SQL injection, and malware, applying real-time quarantine and anomaly logging to PostgreSQL. It filters and forwards only valid messages, enforcing strict rate limits and heartbeat checks. Security modes include open, auth-only, and TLS+ACL. A sniffer, DDos and SQL injector simulate attacks, validating the system's effectiveness in smart building protection.

2. INTRODUCTION

The escalating proliferation of Internet of Things (IoT) devices in recent years has significantly amplified the urgency for robust security measures. IoT devices, ranging from smart home appliances to critical infrastructure components, are increasingly targeted by cyber threats due to their often-inadequate security protocols. A report by Fortinet highlights a dramatic surge in cyber threats powered by artificial intelligence, with automated scans reaching 36,000 per second, and a 500% increase in logs from compromised systems, leading to over 1.7 billion stolen credentials circulating on the dark web.

The inherent vulnerabilities in IoT systems stem from factors such as heterogeneous networking technologies, resource-constrained devices, and often overlooked security and privacy considerations. These weaknesses make IoT systems susceptible to various attacks, including data breaches, unauthorized access, and denial-of-service attacks. Moreover, the lack of standardized security protocols and the rapid deployment of IoT devices without thorough security assessments exacerbate the risk landscape.

The importance of IoT security in today's digital world cannot be overstated. As IoT devices become embedded in daily life and critical sectors—like smart homes, healthcare, and industry—securing them is vital to protect data, privacy, and operations. The IoTBot Security project addresses these challenges by integrating MQTT for lightweight messaging, Docker for modular deployment, and a real-time IoTBot engine for threat detection. It employs HMAC for message integrity, TLS encryption, heartbeat monitoring, and anomaly detection to defend against spoofing, DDoS, and SQL injection. These tools create a proactive, adaptable framework for securing modern IoT ecosystems against evolving cyber threats.

THEORETICAL FRAMEWORK

3. INTERNET OF THINGS (IOT) AND SMART HOME AUTOMATION

The Internet of Things (IoT) refers to the interconnection of physical devices, such as sensors, appliances, and cameras, via the internet to collect, share, and analyze data. For this project, a Middleware-Based approach was utilized, employing an MQTT local server.

The architecture of the IoT system can be structured into five layers:

- **Perception Layer:** Involves data collection using sensors and devices.
- **Network Layer:** Facilitates communication between devices and systems through the internet or local networks.
- **Service Layer:** Manages services that enable efficient communication and interaction between devices and applications. It supports service discovery, composition, and management to optimize data processing and functionality.
- **Application Layer:** Focuses on delivering specific services, such as automation, monitoring, or data analytics.
- **Business Layer:** Handles the business logic and strategies that define the IoT system's value, such as user management, billing, and overall system operation, ensuring that IoT applications are aligned with business objectives.

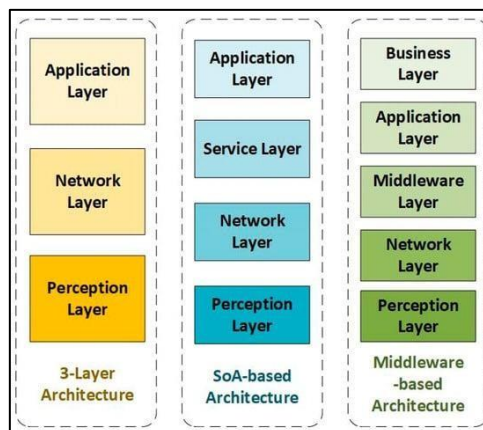


Fig 1 Most common IoT architectures.

4. WIRELESS SENSOR NETWORK

Wireless Sensor Networks (WSNs) consist of a network of spatially distributed sensor nodes designed to monitor various environmental parameters and communicate wirelessly. In the context of smart home automation, WSNs are crucial for enabling real-time data collection, processing, and transmission. The sensor nodes in a WSN are responsible for detecting physical phenomena, such as temperature, humidity, and motion, and transmitting this data to a central processing unit.

in this project consist of distributed, containerized software-based sensor nodes that simulate and transmit environmental data in real time over MQTT. These nodes monitor key parameters such as gas levels, temperature, and motion across multiple building zones. Rather than using hardware like ESP8266 or XBee modules, this system leverages **Dockerized MQTT clients** communicating over a virtual or physical network with a central processing unit—the **Hub**, also containerized. Each sensor node follows a modular architecture with three core components:

- **Sensing Unit:** Simulated in Python, this module generates fake sensing data (e.g., gas concentration, room temperature, or motion state), simulating real-world sensor behavior.
- **Processing Unit:** Embedded in each container, the processing unit formats the data, attaches metadata like timestamps and device IDs, and signs the message using **HMAC** for integrity verification.
- **Communication Unit:** Using **MQTT**, the sensors wirelessly transmit signed data to the central **Hub** over a broker that supports multiple security configurations (open, password-based, TLS with ACLs).

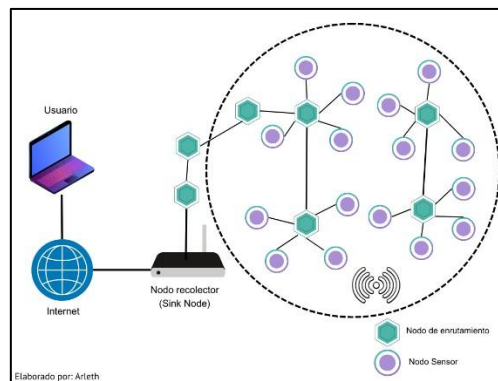


Fig 2 Topology of a WSN

5. COMMUNICATION PROTOCOLS

5.1 Message Queuing Telemetry Transport (MQTT)

The MQTT protocol is a lightweight, efficient messaging protocol designed for IoT applications, particularly in environments where bandwidth is limited, or networks are unreliable. This protocol minimizes bandwidth usage and reduces resource consumption.

It uses a publish/subscribe model for message exchange where clients publish messages on specific topics and subscribe to topics of interest. The elements of the topology of MQTT are the following.

- **Topic:** Is a string that acts as a channel or path for message transmission in the MQTT system.
- **Messages:** Messages are the information you want to exchange between your devices. It can be a command or data.
- **Broker:** Is the central server that manages communication between MQTT clients.
- **Subscriber:** Is a client that subscribes to one or more topics on the MQTT broker.

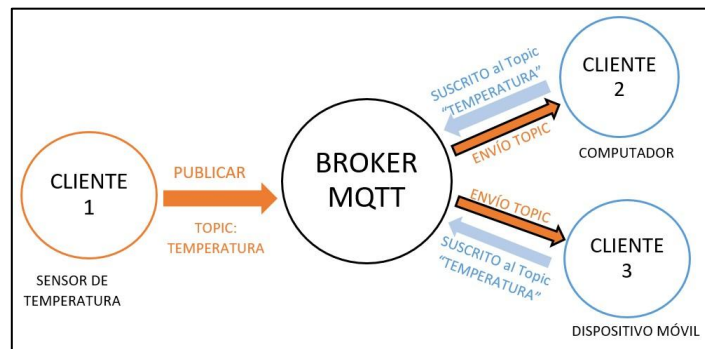


Fig 4 MQTT Topology

MQTT protocol maintains a persistent connection between the client and the server, enabling real-time message delivery. The connection remains open for the lifetime of the client. MQTT does not provide security by itself, but it can be implemented over a secure layer like TLS/SSL to encrypt communication and authenticate clients. Besides, the protocol supports three levels of Quality of Service (QoS) to ensure data delivery according to system requirements:

- **QoS 0:** At most once delivery (best effort).
- **QoS 1:** At least once delivery (ensures message delivery but may result in duplicates).
- **QoS 2:** Exactly once delivery (ensures message is received only once).

6. SOFTWARES

6.1 Docker Engine

It is an open-source containerization technology that enables the development, shipment, and execution of applications in isolated environments known as containers. It provides a lightweight runtime framework for managing these containers, which encapsulates software and its dependencies to ensure consistency across different deployment environments.

The Docker Engine operates through three main components:

1. **Docker Daemon (dockerd)**

This background service manages Docker images, containers, networks, and volumes. It listens for API requests and handles container lifecycle events.

2. **Docker CLI (docker)**

The command-line interface (CLI) allows users to interact with the Docker Daemon using simple terminal commands to build, run, and manage containers.

3. **REST API**

Docker exposes a RESTful API that enables programmatic interaction with the daemon, supporting automation, remote control, and integration with orchestration tools like Docker Compose or Kubernetes.

In the context of the **IoTBot Security** project, Docker Engine is the foundational framework that hosts all components—including sensors, actuators, the hub, MQTT broker, PostgreSQL database, and the IoTBot monitor—as isolated, reproducible containers. This containerized approach improves **portability**, **modularity**, and **scalability**, while also simplifying **deployment across virtual labs (e.g., GNS3)** or physical environments. Each service runs independently but communicates over a shared virtual network, allowing for realistic testing, monitoring, and secure orchestration of the entire IoT ecosystem.

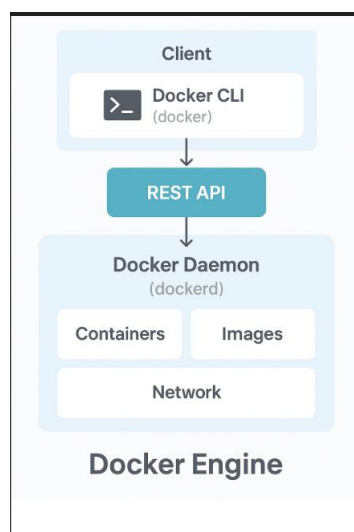


Fig 5. Docker Engine

6.2 GNS3

GNS3 (Graphical Network Simulator-3) is a powerful open-source network emulation platform used to design, simulate, and test complex networks before physical deployment. Unlike traditional simulators, GNS3 allows users to combine **virtual devices** (such as Docker containers, virtual machines, and network OS images) with **real hardware** in a flexible, drag-and-drop graphical interface.

GNS3 consists of two main components:

1. **GNS3 GUI** – The client interface for creating and managing topologies.
2. **GNS3 Server** – Executes the devices and emulates the network, either locally or on a remote server.

In the **IoTBot Security** project, GNS3 is used to emulate a **realistic smart building network** by deploying Dockerized components—such as MQTT sensors, actuators, the central hub, IoTBot, and PostgreSQL—within a virtual network topology. This enables full control over IP addressing, traffic flow, and attack simulation scenarios (e.g., MITM, spoofing, SQL injection), which are difficult to achieve on physical testbeds.

GNS3 supports:

- **Docker integration:** Run each sensor, actuator, or security bot as an isolated container node.
- **Custom links and interfaces:** Simulate LAN, Wi-Fi, or internet environments.
- **Capture tools (Wireshark, Scapy):** Monitor traffic and analyze security behaviors.
- **Remote/cloud execution:** Scale test environments using remote GNS3 servers or virtualization platforms.

By using GNS3, the project achieves **repeatable experiments, realistic threat modeling, and network-aware deployment testing**. This framework bridges the gap between abstract simulation and real-world network implementation, making it ideal for advanced IoT cybersecurity prototyping.

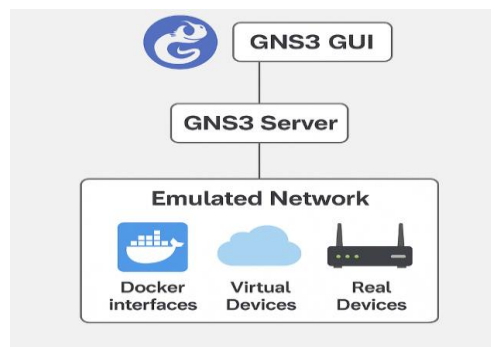


Fig 6. GNS3 Emulator

6.3 Postgres-SQL

PostgreSQL is a powerful, open-source relational database management system known for its robustness and advanced feature set. Unlike lighter systems like SQLite, PostgreSQL runs as a dedicated server process and supports complex queries, ACID transactions, and concurrency control. It is designed to handle large volumes of structured data and is commonly used in enterprise applications, web services, and systems requiring high reliability. In the **IoTBot Security** project, PostgreSQL acts as the backend database, storing sensor data, actuator responses, registered devices, and security anomalies, making it ideal for applications that demand persistent, secure, and queryable storage.



Fig 7. Postgres-SQL Database

6.4 Mosquitto

Eclipse Mosquitto is an open-source message broker that implements MQTT protocol versions 5.0 and 3.1. Mosquitto is lightweight and suitable for use on all devices, from low-power single-board computers to full servers. This broker works with the MQTT protocol, providing a lightweight method for sending messages using a publish/subscribe model. This makes it suitable for Internet of Things messaging, such as with low-power sensors or mobile devices like phones, embedded computers, or microcontrollers.

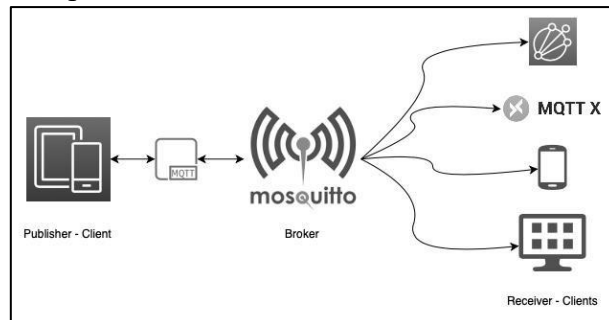


Fig 8 Mosquitto Broker

METHODOLOGICAL FRAMEWORK

7. IMPLEMENTATION

7.1 General Topology

As shown in the figure, the architecture of the **IoTBot Security** system is composed of several interconnected Docker-based components emulated in **GNS3**. The core of the network is built around a virtual switch (**Switch1**) that interconnects all devices in a controlled virtual environment. Each device is deployed as a Docker container and assigned to specific functional zones: sensors, actuators, control, security, servers, and adversarial components.

The **sensor zone** includes three virtual MQTT clients: gas-sensor-x-1, motion-sensor-x-1, and temp-sensor2x-1. These simulate physical environmental sensors and publish data over MQTT to a centralized broker (mqtt-broker-1). The **actuator zone** features hvac-x-1, door-x-1, and alarm-x-1, each waiting for control commands from the central hub-x-1, which performs real-time decision-making and data logging to postgres-db-1.

The hub-x-1 acts as the brain of the systems subscribing to filtered MQTT topics, validating message authenticity with HMAC, and forwarding commands to actuators when thresholds (e.g., gas levels, motion) are crossed. This hub is further secured by iotbot-x-1, a custom-built anomaly detection engine that monitors all MQTT traffic, identifies spoofing, DDoS, and SQL injection attempts, and logs malicious activity in real time.

The **server zone** hosts the mqtt-broker-1 (Mosquitto) and postgres-db-1 for database logging; both connected to a NAT interface for potential cloud communication. Additionally, a Kali-GUI-1 node in the **hacker zone** simulates attacker behavior, testing the system's defenses through packet sniffing, spoofing, and injection attempts.

This general topology emulates a **secure smart building environment**, with all components communicating over MQTT, orchestrated and visualized using GNS3, and defended through modular Docker isolation and real-time IoT threat detection.

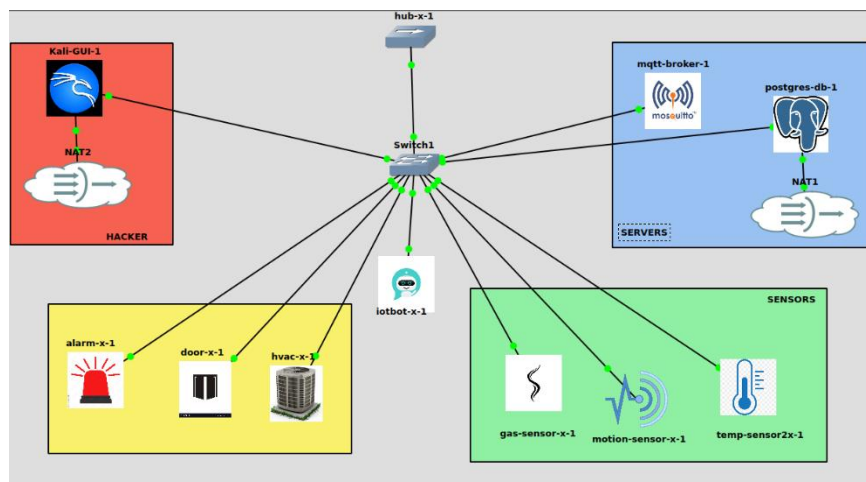


Fig 9 Topology of devices connected

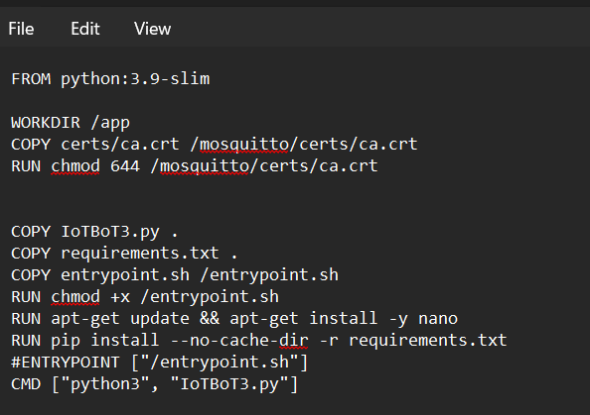
7.2 Docker Configuration

In this project, Docker was used to containerize each component—sensors, actuators, central hub, MQTT broker, database, and the IoTBot security agent—ensuring portability, isolation, and consistent behavior across development and testing environments. Docker containers were built using `Dockerfile` scripts, which define all the necessary steps to assemble an application into a self-contained and executable image.

A **Dockerfile** is a set of sequential instructions that tells Docker how to build the environment needed to run a specific component. It typically includes a base image (like Python), installation of system packages, Python dependencies, copying application files, and setting permissions.

As shown in the figure 10, the Dockerfile used to build the **IoTBot** container starts from a lightweight base image: `python:3.9-slim`. It sets the working directory to `/app`, copies the necessary certificate authority file (`ca.crt`) into the appropriate directory for MQTT/TLS authentication, and sets file permissions. Then, it copies the `IoTBot3.py` script, the Python dependency list (`requirements.txt`). File permissions are configured to ensure the entry point script is executable. It also installs useful tools like `nano` for internal debugging. Finally, it installs the required Python packages using `pip` and defines the main command to execute the security bot using `CMD ["python3", "IoTBot3.py"]`.

The same Docker creation pattern is applied across all components in the project to ensure consistency: each service is encapsulated in its own Docker image, built from a Dockerfile with its dependencies, logic, and startup command. This approach simplifies deployment in GNS3 and guarantees reproducibility across different environments.



```
File Edit View

FROM python:3.9-slim

WORKDIR /app
COPY certs/ca.crt /mosquitto/certs/ca.crt
RUN chmod 644 /mosquitto/certs/ca.crt

COPY IoTBot3.py .
COPY requirements.txt .
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh
RUN apt-get update && apt-get install -y nano
RUN pip install --no-cache-dir -r requirements.txt
#ENTRYPOINT ["/entrypoint.sh"]
CMD ["python3", "IoTBot3.py"]
```

Fig 10 IoT Security BoT

7.3 General Settings

The general configuration of the **IoTBot Security** system is structured around a highly modular, environment-variable-driven Python script. This configuration logic ensures portability, flexibility, and security when deploying across different Docker containers in the GNS3 emulated network.

As shown in the figures 11a and 11b, the configuration segment of the `IoTBot3.py` script begins by importing essential Python libraries for secure MQTT communication (`paho.mqtt.client`), PostgreSQL database access (`psycopg2`), cryptographic validation

(`hmac`, `hashlib`), and time handling. These imports establish the foundational dependencies required for secure and autonomous behavior across the IoT network.

All critical runtime values are dynamically loaded using `os.getenv()`. This allows for **environment-based configuration**, avoiding hardcoded secrets and improving container portability. Parameters include the MQTT broker address (`BROKER`), the appropriate port (1883 or 8883 depending on whether TLS is enabled), user authentication credentials (`MQTT_USERNAME`, `MQTT_PASSWORD`), and the shared HMAC secret for message integrity. The activation of **TLS**, **authentication**, and **HMAC verification** is controlled by environment flags (`ENABLE_TLS`, `ENABLE_AUTH`, and `ENABLE_HMAC` respectively), making the system easily adaptable to different security scenarios.

Database configuration is handled via the `DB_PARAMS` dictionary, which contains PostgreSQL connection details such as host, port, user, password, and database name. This enables the IoTBot to log detected anomalies and device actions securely and efficiently in a centralized storage backend.

Additionally, the `RAW_TOPICS` list defines the unfiltered MQTT topics the IoTBot listens to—covering temperature, motion, gas, and device registration. The `FORWARD_MAP` dictionary defines how these raw messages are remapped to filtered topics, ensuring that only verified and safe data is forwarded to the hub and actuators.

Furthermore, all other Docker containers in the **IoTBot Security** system—including the motion sensor, gas sensor, temperature sensor, actuators, and follow this same configuration model. Each service uses `os.getenv()` to pull its MQTT, TLS, and authentication parameters, ensuring consistency and modularity. Some components also introduce additional libraries depending on their roles. For example, the motion sensor uses the `faker` library to simulate realistic activity, while actuators verify HMACs before performing actions. Despite slight variations in libraries and behavior, all containers adopt this shared configuration logic to maintain system integrity and simplify deployment and debugging.

```
import os
import json
import ssl
import hmac
import hashlib
import time
import paho.mqtt.client as mqtt
import psycogp2
from datetime import datetime, timezone
from collections import defaultdict

# --- CONFIG ---
BROKER = os.getenv("BROKER_ADDRESS", "mqtt-broker")
PORT = 8883 if os.getenv("ENABLE_TLS", "false").lower() == "true" else 1883
USERNAME = os.getenv("MQTT_USERNAME", "")
PASSWORD = os.getenv("MQTT_PASSWORD", "")
HMAC_SECRET = os.getenv("HMAC_SECRET", "123iot45").encode()
TLS = os.getenv("ENABLE_TLS", "false").lower() == "true"
AUTH = os.getenv("ENABLE_AUTH", "false").lower() == "true"
ENABLE_HMAC = os.getenv("ENABLE_HMAC", "true").lower() == "true"
```

Fig 11a General Settings

```

DB_PARAMS = {
    "host": os.getenv("DB_HOST", "postgres_db"),
    "port": os.getenv("DB_PORT", "5432"),
    "user": os.getenv("DB_USER", "postgres"),
    "password": os.getenv("DB_PASSWORD", "postgres"),
    "dbname": os.getenv("DB_NAME", "iot_logs")
}

RAW_TOPICS = [
    "building/zone2/temperature/room1",
    "building/zone1/motion/entrance",
    "building/zone3/gas/detection",
    "device/registration"
]

FORWARD_MAP = {
    "building/zone2/temperature/room1": "building/filtered/zone2/temperature",
    "building/zone1/motion/entrance": "building/filtered/zone1/motion",
    "building/zone3/gas/detection": "building/filtered/zone3/gas"
}

```

Fig 11b General Settings

7.4 MQTT Setup

In the **IoTBot Security** system, the MQTT communication layer is powered by a customized and hardened Docker container running the **Eclipse Mosquitto 2.0** broker. This container was built from scratch using a Dockerfile, password generation script, and a dynamic entrypoint script that configures the broker securely based on selected security modes.

The Dockerfile begins by pulling the official `eclipse-mosquitto:2.0` base image and switches to root to perform system-level operations. It removes the default broker configuration and sets up custom directories and ownerships for configuration and certificate management. It copies in necessary files: the configuration directory (`config/`), certificate files (`certs/`), a password generation script (`gen_passwordfile.sh`), and the dynamic broker entrypoint script (`entrypoint.sh`). File permissions are adjusted for security: certificate files are restricted to read-only, while the ACL and password files are tightly locked down using `chmod 600` and `chmod 700`.

```

FROM eclipse-mosquitto:2.0
USER root

# Remove default config
RUN rm -f /mosquitto/config/mosquitto.conf
RUN usermod -d /home/mosquitto -s /bin/sh mosquitto && \
    mkdir -p /home/mosquitto && \
    chown mosquitto:mosquitto /home/mosquitto
COPY config/ /mosquitto/config/
COPY certs/ /mosquitto/certs/
COPY gen_passwordfile.sh /gen_passwordfile.sh
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /gen_passwordfile.sh /entrypoint.sh
RUN chown -R mosquitto:mosquitto /mosquitto/certs && \
    chmod 644 /mosquitto/certs/*.crt && \
    chmod 640 /mosquitto/certs/server.key
RUN /gen_passwordfile.sh && \
    chown mosquitto:mosquitto /mosquitto/config/passwordfile /mosquitto/config/aclfile && \
    chmod 600 /mosquitto/config/passwordfile && \
    chmod 700 /mosquitto/config/aclfile
USER mosquitto
EXPOSE 8883
ENTRYPOINT ["/entrypoint.sh"]

```

Fig 12 DockerFile MQTT-BROKER

The `gen_passwordfile.sh` script runs during the build phase and uses `mosquitto_passwd` to automatically generate a secure password file. Four users are pre-created with dedicated credentials: `actuator_user`, `sensor_user`, `hub_user`, and `iotbot_user`. These are stored in `/mosquitto/config/passwordfile`.

The final piece is the `entrypoint.sh`, which acts as a **dynamic configurator**. When the container starts, it:

Checks if TLS certificates are present and fixes permissions if needed.

Reads the `ENABLE_TLS` and `ENABLE_AUTH` environment variables to decide which configuration mode to activate.

Depending on the combination, it copies one of four predefined configuration files into place:

- `mosquitto_tls_auth.conf`
- `mosquitto_tls_only.conf`
- `mosquitto_auth_only.conf`
- `mosquitto_open.conf`

Finally, it prints the selected configuration and launches the broker.

This setup provides a flexible, **multi-mode MQTT broker**, supporting:

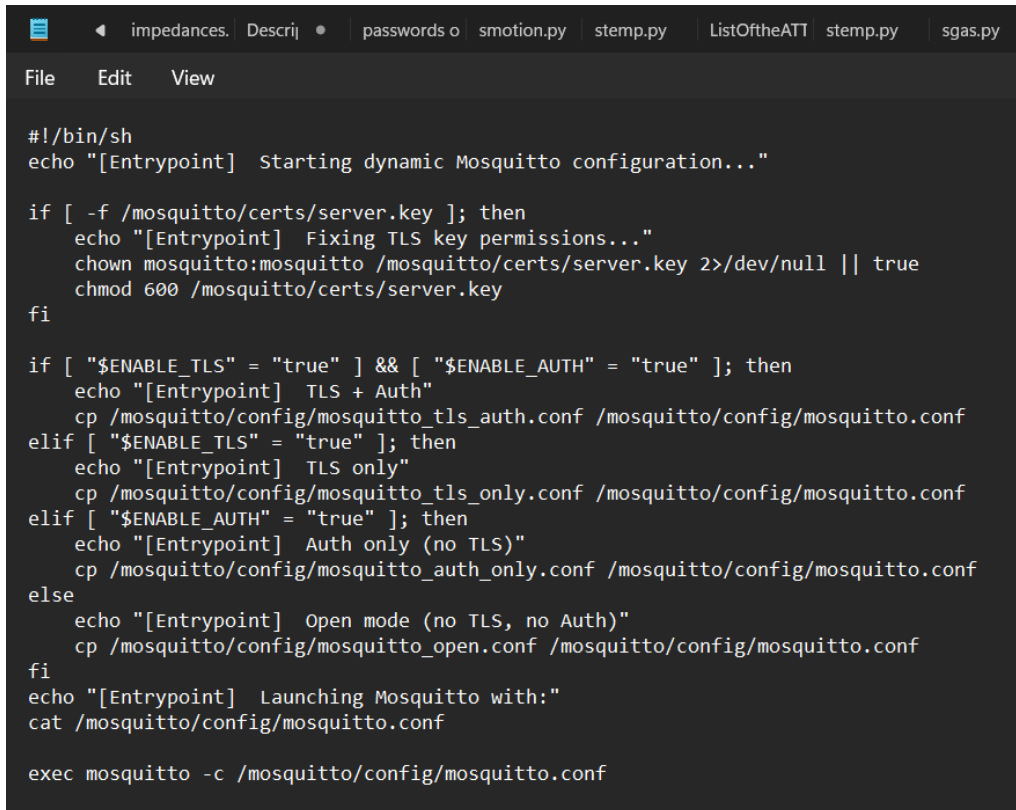
Open mode (no auth, no TLS)

Authentication only

TLS only

TLS with username/password and ACLs

This Docker-based Mosquitto container can be reused across any system where secure, modular, and customizable MQTT communication is required—making it ideal for the evolving needs of IoT security.

A screenshot of a terminal window with a dark background. The terminal shows a shell script for the Mosquitto container's entrypoint. The script starts with a shebang and an echo statement. It then uses conditional logic to check for the presence of a server key and to configure Mosquitto based on environment variables for TLS and authentication. The script ends with an exec command to launch Mosquitto.

```
#!/bin/sh
echo "[Entrypoint] Starting dynamic Mosquitto configuration..."

if [ -f /mosquitto/certs/server.key ]; then
    echo "[Entrypoint] Fixing TLS key permissions..."
    chown mosquitto:mosquitto /mosquitto/certs/server.key 2>/dev/null || true
    chmod 600 /mosquitto/certs/server.key
fi

if [ "$ENABLE_TLS" = "true" ] && [ "$ENABLE_AUTH" = "true" ]; then
    echo "[Entrypoint] TLS + Auth"
    cp /mosquitto/config/mosquitto_tls_auth.conf /mosquitto/config/mosquitto.conf
elif [ "$ENABLE_TLS" = "true" ]; then
    echo "[Entrypoint] TLS only"
    cp /mosquitto/config/mosquitto_tls_only.conf /mosquitto/config/mosquitto.conf
elif [ "$ENABLE_AUTH" = "true" ]; then
    echo "[Entrypoint] Auth only (no TLS)"
    cp /mosquitto/config/mosquitto_auth_only.conf /mosquitto/config/mosquitto.conf
else
    echo "[Entrypoint] Open mode (no TLS, no Auth)"
    cp /mosquitto/config/mosquitto_open.conf /mosquitto/config/mosquitto.conf
fi
echo "[Entrypoint] Launching Mosquitto with:"
cat /mosquitto/config/mosquitto.conf

exec mosquitto -c /mosquitto/config/mosquitto.conf
```

Figure 13 EntryPoint for Dynamic MQTT-Configuration

A dedicated **Access Control List (ACL)** file is used to define fine-grained topic-level permissions for each authenticated MQTT user as shown in figure 14 and 15 . This ACL ensures that users can only publish or subscribe to topics relevant to their roles in the system. For instance, `actuator_user` is permitted to write to state topics and read control topics for each actuator, while `sensor_user` is only allowed to publish raw sensor data. The `iotbot_user` can read all raw topics and write to filtered ones, serving as the security gatekeeper, and `hub_user` can subscribe to filtered topics and issue commands to actuators. This role-based access strategy enforces **strict isolation and secure MQTT communication**, preventing unauthorized devices from accessing or manipulating critical topics

```
# ——— ACTUATOR USERS ———
user actuator_user
topic write building/zone2/ac/state
topic read  building/zone2/ac/control

topic write building/zone1/door/state
topic read  building/zone1/door/lock

topic write building/zone3/alarm/state
topic read  building/zone3/alarm/control

topic readwrite hub/heartbeat/#
# ——— SENSOR USERS ———
user sensor_user
topic write building/zone2/temperature/#
topic write building/zone1/motion/#
topic write building/zone3/gas/#
topic readwrite hub/heartbeat/#
```

Fig 14 Access list Actuators and Sensors

```
listener 8883 0.0.0.0

cafile /mosquitto/certs/ca.crt
certfile /mosquitto/certs/server.crt
keyfile /mosquitto/certs/server.key

allow_anonymous false
password_file /mosquitto/config/passwordfile
acl_file /mosquitto/config/aclfile
```

Fig 15 MQTT setup TLS+AUTH

7.5 Sensor Configuration

Each sensor node in the **IoTBot Security** system is implemented as an independent Docker container, designed to simulate real-time environmental data. These sensors publish messages to the MQTT broker using a secure, heartbeat-aware publishing loop.

As illustrated in figure 16, the core of each sensor operates within an infinite while True loop that sends data periodically every 5 seconds. Before publishing, the sensor checks if the central hub has sent a heartbeat within the last 10 seconds. If so, the sensor generates a new temperature value using `random.uniform()` within a realistic range (19.0°C to 25.0°C). The generated reading is then packed into a JSON-compatible dictionary (pkt) containing the `device_id`, sensor type, value, and timestamp.

```

while True:
    if time.time() - last_hb <= 10:
        temp = round(random.uniform(19.0, 25.0), 2)
        pkt = {
            "device_id": DEVICE_ID,
            "sensor": "temperature",
            "value": temp,
            "timestamp": now_str()
        }
        if ENABLE_HMAC:
            pkt["hmac"] = sign(pkt)
        client.publish(TOPIC, json.dumps(pkt))
        print(f"[TEMP SENSOR] Temp: {temp} °C")
    else:
        print("[TEMP SENSOR] Paused (hub offline)")
        time.sleep(5)

```

Fig 16 Temperature Sensor Generating fake data.

Crucially, the second image shows how the `sign(pkt)` function ensures message integrity using **HMAC (Hash-Based Message Authentication Code)**. This function serializes the packet content in a deterministic way (sorted keys, fixed separators) and generates a SHA-256 hash using a shared secret (`HMAC_SECRET`). If HMAC is enabled (`ENABLE_HMAC = true`), this hash is embedded in the message as a new field (`pkt["hmac"]`), allowing the receiving component to verify authenticity and integrity.

```

def sign(pkt):
    if not ENABLE_HMAC:
        return ""
    raw = json.dumps({k: pkt[k] for k in sorted(pkt)}, separators=(",", ":"), sort_keys=True).encode()
    return hmac.new(HMAC_SECRET, raw, hashlib.sha256).hexdigest()

```

Fig 17 Computing HMAC Function Signing.

If the hub becomes unreachable (i.e., no recent heartbeat), the sensor pauses all publishing activity, ensuring that stale or unsupervised data is not pushed to the broker. This **heartbeat mechanism**, combined with **HMAC signing**, enforces a secure and responsive design—preventing spoofing, minimizing noise, and ensuring only verified, fresh data is accepted into the system.

This configuration pattern is applied uniformly across all sensor types (temperature, gas, motion), ensuring consistent security and communication logic across the entire IoT network.

7.6 Actuator Configuration

The HVAC actuator in the **IoTBot Security** project is designed to securely receive temperature data and autonomously respond by adjusting its operational state. It uses **HMAC (Hash-based Message Authentication Code)** to ensure the integrity and authenticity of incoming control messages and outgoing state reports.

As shown in **Figure 18**, the HVAC script implements two key functions: `sign(pkt)`, previously introduced in the **Sensor Section (Figure 17)**, and `verify(pkt)`. The `verify(pkt)` function plays a critical role in message authentication. It retrieves the `hmac` field from the received payload, reconstructs the original data excluding the HMAC itself, and computes a new hash using the shared secret. If the recalculated hash matches the one received, the message is deemed authentic. Otherwise, it is rejected as potentially spoofed or altered—ensuring that only trusted and untampered messages are processed by the actuator.

```
def verify(pkt):
    if not ENABLE_HMAC:
        return True
    sig = pkt.get("hmac")
    data = {k: pkt[k] for k in sorted(pkt) if k != "hmac"}
    calc = hmac.new(HMAC_SECRET, json.dumps(data, separators=(',', ':'), sort_keys=True).encode(), hashlib.sha256).hexdigest()
    return sig and hmac.compare_digest(calc, sig)
```

Fig 18 Verify HMAC Function

Figure 18 highlights the `on_message()` function, where the HVAC listens for control messages on a predefined MQTT topic. Upon receiving a message, the HMAC signature is validated before any further action is taken. If valid, the new temperature value is extracted, and a decision is made: if the temperature is below 20°C, the HVAC enters heating mode; above 24°C, it switches to cooling; otherwise, it remains idle. If the state changes, the HVAC publishes its new status—again signed with HMAC for security.

```
def on_message(client, userdata, msg):
    global last_heartbeat, current_temp, current_state
    if msg.topic == HEARTBEAT_TOPIC:
        last_heartbeat = time.time()
    elif msg.topic == CONTROL_TOPIC:
        try:
            data = json.loads(msg.payload.decode())
            if not verify(data):
                print("[HVAC] Invalid HMAC - rejected.")
                return

            new_temp = data.get("value")
            if new_temp is not None:
                current_temp = new_temp
                print(f"[HVAC] Sensor reports: {current_temp}°C")

            new_state = current_state
            if current_temp < 20.0:
                new_state = "heating"
            elif current_temp > 24.0:
                new_state = "cooling"
            else:
                new_state = "idle"

            if new_state != current_state:
                current_state = new_state
                publish_state(client)
        except Exception as e:
            print(f"[HVAC] Error: {e}")
```

Fig 18 On Message Function for HVAC actuator

Importantly, **this exact HMAC-based logic is applied across all actuator components** in the system, including the smart door lock and gas alarm. Each actuator validates the authenticity of MQTT commands using `verify()`, acts only on secure messages, and publishes signed feedback using `sign()`. This consistent use of message integrity verification ensures that no actuator can be hijacked or manipulated by unauthorized sources, reinforcing a system-wide trust model that aligns with IoT security best practices.

7.7 HUB Configuration

The **hub** in the *IoTBot Security* system serves as the **central logic unit**, responsible for interpreting sensor data, issuing control commands to actuators, logging activity to the database, and maintaining synchronized communication across the MQTT infrastructure. Its design integrates secure communication, real-time decision-making, and heartbeat supervision, forming the coordination core of the entire IoT architecture.

As seen in figure 19, the hub includes a `log_to_db()` function that connects to the PostgreSQL database using parameters defined in `DB_PARAMS`. It inserts incoming messages into either the `sensor_data` or `actuator_actions` table with a timestamp localized to the Europe/Rome timezone. This function ensures persistent logging of all important data events, aiding future forensic analysis and auditing.

```
def log_to_db(table, device_id, payload):
    try:
        conn = psycopg2.connect(**DB_PARAMS)
        cursor = conn.cursor()
        timestamp = datetime.now(ZoneInfo("Europe/Rome"))
        cursor.execute(
            f"INSERT INTO {table} (device_id, payload, timestamp) VALUES (%s, %s, %s)",
            (device_id, json.dumps(payload), timestamp)
        )
        conn.commit()
        cursor.close()
        conn.close()
    except Exception as e:
        print(f"[Hub] DB log failed: {e}")
```

Fig 19 Recording data on Postgres DB.

Figure 20 shows `on_message()` handler, which defines the hub's core behavior. Upon receiving a message, it first verifies the HMAC signature using `verify_hmac()` to ensure authenticity. If the message is valid and matches a known topic (e.g., `building/filtered/zone1/motion`), the hub evaluates the sensor state and generates a corresponding control command. For example, detecting motion results in an unlock command sent to the smart door actuator, same logic for HVAC and Alarm. The message is signed before being published to preserve end-to-end integrity `compute_hmac()`.

```

def on_message(client, userdata, msg):
    try:
        payload = json.loads(msg.payload.decode())
        if not verify_hmac(payload):
            print("[Hub] Invalid HMAC – rejected.")
            return

        device_id = payload.get("device_id", "unknown")
        topic = msg.topic
        payload.pop("hmac", None)

        # === MOTION
        if topic == "building/filtered/zone1/motion":
            state = payload.get("value")
            if sensor_states.get(topic) != state:
                sensor_states[topic] = state
                action = "unlock" if state == "motion_detected" else "lock"
                cmd = {"action": action, "device_id": "hub", "timestamp": get_timestamp()}
                cmd["hmac"] = compute_hmac(cmd)
                client.publish("building/zone1/door/lock", json.dumps(cmd))
                print(f"[Hub] Door command: {action.upper()}")

```

Fig 20 On Message HUB.

Figure 21 illustrates the **heartbeat loop**, which runs in parallel to continuously broadcast the hub's status to the network. Every few seconds (controlled by HEARTBEAT_INTERVAL), the hub publishes a message to the hub/heartbeat topic, including a timestamp and HMAC signature. This signal allows other devices (sensors and actuators) to verify that the hub is alive and responsive. If the heartbeat is missing for a defined duration, sensors and actuators automatically pause to avoid unsupervised operation.

```

while True:
    hb = {"hub_status": "alive", "timestamp": get_timestamp(), "device_id": "hub"}
    hb["hmac"] = compute_hmac(hb)
    client.publish(HEARTBEAT_TOPIC, json.dumps(hb))
    time.sleep(HEARTBEAT_INTERVAL)

```

Fig 21 HeartBeat Procedure

7.8 IoT Security BoT

The **IoTBot** component plays a central role in enforcing security policies, detecting malicious behavior, and maintaining data integrity across the entire **IoTBot Security** system. Acting as a real-time MQTT gatekeeper, the bot analyzes all incoming messages from sensors and devices, filters out anomalies, logs suspicious behavior, and only forwards validated data to the central hub.

7.8.1 Device Verification and Access Control

As shown in figure 22, IoTBot maintains a strict list of KNOWN_DEVICES and a set of ALLOWED_KEYS to define valid message structures. This prevents rogue devices or

malformed payloads from interacting with the system. Unknown devices are monitored via a rate-limited registration process and stored in a temporary quarantine list (QUARANTINE) if suspicious behavior is detected.

```
KNOWN_DEVICES = {"sensor_motion_01", "sensor_temp_01", "sensor_gas_01", "door_lock_01", "hvac_01", "alarm_gas_01", "hub"}
RATE_LIMIT = defaultdict(list)
QUARANTINE = set()
ALLOWED_KEYS = {"device_id", "sensor", "value", "state", "timestamp", "hmac", "hub_status", "action"}
```

Fig 22 Allowed Keys and Devices

7.8.2 DDoS Detection and Global Flood Control

To safeguard the MQTT infrastructure from overload and malicious traffic bursts, the **IoTBot Security** system implements a robust, multi-tiered **DDoS detection and flood control mechanism**. This mechanism evaluates the frequency and origin of messages and applies tailored mitigation based on the scale and type of threat.

The system uses **three core detection functions**, each designed to identify a specific DDoS behavior:

- **Global Flood Detection (is_global_ddos()):**

This function monitors the total volume of MQTT messages arriving from all devices. If more than **100 messages are received across the system within 10 seconds**, it flags a system-wide DDoS attempt and **enables GLOBAL_DDOS_MODE**. While this mode is active, all unknown or unregistered devices are automatically ignored to reduce network stress.

This function is called inside the **on_message handler**, which is triggered every time MQTT message is received regardless of the topic. So **as long as the IoTBot subscribes to # (all topics)**, and on_message() is active, is_global_ddos() automatically tracks **every message** received — across **all topics**.

```
NEW_DEVICE_WINDOW = 30
NEW_DEVICE_LIMIT = 10
NEW_DEVICES_LOG = []

DDOS_GLOBAL_WINDOW = 10
DDOS_GLOBAL_MAX = 100
GLOBAL_FLOOD_LOG = []

GLOBAL_DDOS_MODE = False

def is_global_ddos():
    now = time.time()
    GLOBAL_FLOOD_LOG[:] = [t for t in GLOBAL_FLOOD_LOG if now - t < DDOS_GLOBAL_WINDOW]
    GLOBAL_FLOOD_LOG.append(now)
    return len(GLOBAL_FLOOD_LOG) > DDOS_GLOBAL_MAX
```

Fig 23 Global Flood Detection DDOS

- **New Device Surge Detection (is_ddos_from_new_ids()):**

This function detects if a large number of **new, previously unseen device IDs** start publishing messages in a short timeframe—typically a sign of a distributed bot attack. If **more than 10**

new devices attempt to publish within 30 seconds, each one is added to a **quarantine list**, and their traffic is blocked.

```
def is_ddos_from_new_ids(device_id):
    now = time.time()
    NEW_DEVICES_LOG[:] = [t for t in NEW_DEVICES_LOG if now - t[0] < NEW_DEVICE_WINDOW]
    if device_id not in KNOWN_DEVICES and device_id not in QUARANTINE:
        NEW_DEVICES_LOG.append((now, device_id))
    ids = {d for _, d in NEW_DEVICES_LOG}
    return len(ids) > NEW_DEVICE_LIMIT
```

Fig 24 New Device Surge Detection

Per-Device Rate Limiting (RATE_LIMIT):

This mechanism tracks how frequently each individual device sends messages. If any device sends **more than 50 messages within a 10-second window**, it is quarantined to prevent it from overloading the broker, regardless of whether it is known or unknown.

These functions are evaluated **in real-time** for every message that passes through the IoTBot. If a message violates any of the defined thresholds, the device is either blocked, quarantined, or logged for further inspection. This ensures that the system remains responsive, secure, and resilient under various types of DDoS pressure—whether caused by one rogue device or a wave of coordinated attackers.

```
RATE_LIMIT[device_id] = [t for t in RATE_LIMIT[device_id] if t > time.time() - 10]
RATE_LIMIT[device_id].append(time.time())
if len(RATE_LIMIT[device_id]) > 50:
    QUARANTINE.add(device_id)
    log_anomaly(device_id, {"event": "ddos_rate_limit", "payload": data})
    return

if device_id in KNOWN_DEVICES:
    register_device(device_id)
```

Fig 25 Per-Device Rate Limiting

Table 1 Summary of DDOS Detection Mechanism

Guard	Target	Trigger	Action
is_global_ddos()	All devices	More than 100 messages in 10 seconds	Enable GLOBAL_DDOS_MODE
is_ddos_from_new_ids()	Unknown Devices	More than 10 new device IDs in 30 seconds	Quarantine new device
RATE_LIMIT()	Individual device	More than 50 messages in 10 seconds	Quarantine the specific device

This layered DDoS protection model allows the IoTBot to identify not only brute-force flooding but also stealthy or distributed attacks. Each layer enforces a different dimension of control—system-wide, identity-based, and frequency-based—ensuring comprehensive protection against MQTT exploitation and broker exhaustion.

7.8.3 SQL Injection and Malware Injection

The IoTBot runs deep payload inspection. The `is_sqli_like()` function scans device IDs for typical injection patterns (quotes, semicolons, SQL commands), while `is_malware()` ensures that only whitelisted keys exist in incoming payloads. Both mechanisms proactively isolate compromised or malicious nodes.

```
def is_sqli_like(device_id):
    suspicious = ['"', "'", ";", "--", "drop", "insert", "update", "delete"]
    return any(x in device_id.lower() for x in suspicious)

def is_malware(payload_dict):
    return any(k not in ALLOWED_KEYS for k in payload_dict)
```

Fig 26 Global Flood Detection DDoS

7.8.4 Logging and Registration

As shown, every anomaly is persistently stored in the PostgreSQL database using the `log_anomaly()` function. This allows full traceability of incidents including spoofed packets, SQLi attempts, excessive messaging, and unauthorized access. Devices are also registered to a `registered_devices` table, and duplicates are avoided using PostgreSQL's `ON CONFLICT DO NOTHING`.

```
def is_sqli_like(device_id):
    suspicious = ['"', "'", ";", "--", "drop", "insert", "update", "delete"]
    return any(x in device_id.lower() for x in suspicious)

def is_malware(payload_dict):
    return any(k not in ALLOWED_KEYS for k in payload_dict)
```

Fig 27 Global Flood Detection DDoS

Real-Time MQTT Message Handling

The `on_message()` handler orchestrates the bot's real-time behavior. Upon receiving a message:

- It parses and validates the JSON payload
- Runs all layered checks (DDoS, HMAC, SQLi, malware)
- Logs and quarantines as needed
- Forwards valid messages from known devices to the filtered topic defined in `FORWARD_MAP`
- Drops or logs others

Each forwarded message is printed and tracked to maintain transparency of the bot's operations.

```
def on_message(client, userdata, msg):
    global GLOBAL_DDOS_MODE
    try:
        data = json.loads(msg.payload.decode())
        device_id = data.get("device_id") or data.get("sensor")

        # === GLOBAL DDoS CHECK ===
        if is_global_ddos():
            GLOBAL_DDOS_MODE = True
            print("[IoTBot] GLOBAL DDoS MODE TRIGGERED")

        # === Smart Mitigation ===
        if GLOBAL_DDOS_MODE and device_id not in KNOWN_DEVICES:
            print(f"[IoTBot] {device_id} dropped (DDoS mode)")
            log_anomaly(device_id, {"event": "ddos_mode_drop", "payload": data})
            return

        if is_ddos_from_new_ids(device_id):
            QUARANTINE.add(device_id)
            log_anomaly(device_id, {"event": "new_device_ddos", "payload": data})
            return

        if device_id in QUARANTINE:
            log_anomaly(device_id, {"event": "quarantined", "payload": data})
            return

        if msg.topic == "device/registration" and device_id not in KNOWN_DEVICES:
            QUARANTINE.add(device_id)
            log_anomaly(device_id, {"event": "unauthorized_registration", "payload": data})
            return

        if not verify_hmac(data):
            QUARANTINE.add(device_id)
```

Fig 27 On Message Function IoTSecBoT

IoTBot architecture exemplifies a **modular and reactive cybersecurity layer** in an IoT system. It combines traffic monitoring, cryptographic verification, behavioral analysis, and persistent forensic logging to deliver an active defense engine capable of **identifying, reacting to, and isolating threats** in real time

7.9 TLS on MQTT

To ensure secure communication between MQTT clients and the broker, the system implements **Transport Layer Security (TLS)** by generating its own **public key infrastructure (PKI)**, including a **Certificate Authority (CA)** and **broker-side certificates**. This setup enables **end-to-end encryption**, preventing unauthorized interception or tampering of MQTT messages.

The TLS generation process is fully automated through a shell script, as shown in the implementation in figure 29. The script performs the following steps:

1. Configuration Setup

A Subject Alternative Name (SAN) configuration file named `san-ip.cnf` is created dynamically, specifying the broker's IP address (192.168.10.1) and necessary TLS extensions such as `serverAuth`. This ensures compatibility with modern clients that enforce SAN validation.

2. **CA Generation**

A private key (`ca.key`) is created for the Certificate Authority, followed by the generation of a self-signed CA certificate (`ca.crt`). This CA will be used to sign the MQTT broker's certificate.

3. **Broker Certificate and CSR**

A private key for the broker (`server.key`) is generated, along with a Certificate Signing Request (`server.csr`) using the SAN config. This CSR includes metadata and the broker's identity, which will be certified by the CA.

4. **Certificate Signing**

The broker certificate (`server.crt`) is signed by the CA using `openssl x509`. It includes SAN support and is valid for 365 days. The resulting `.crt` and `.key` files are used by Mosquitto to serve TLS-secured MQTT connections.

5. **Cleanup and Output**

Temporary files are removed, and the script prints out the final TLS assets: `ca.crt`, `server.crt`, and `server.key`.

6. By using **self-signed certificates with SAN support**, this process enables secure MQTT sessions without relying on external certificate providers. All MQTT clients are configured to trust the CA (`ca.crt`) and validate the broker's certificate during TLS handshakes. This ensures **confidentiality**, **authenticity**, and **integrity** of all MQTT messages exchanged within the system.

7.9.1 **Workflow TLS**

8. **Broker sends `server.crt` (includes public key + CA signature)**

1. **Client uses `ca.crt`** to verify the certificate is trusted
2. **Client encrypts a secret** using broker's public key
3. **Broker decrypts it** with `server.key`
4. If successful → client knows the broker is authentic

TLS tunnel established → all MQTT messages are now encrypted

```

IP="192.168.10.1"
DAYS=365
CN="MQTT Broker"

# === SAN CONFIG FILE ===
SAN_CONFIG="san-ip.cnf"

cat > $SAN_CONFIG <<EOF
[req]
distinguished_name = req_distinguished_name
req_extensions = v3_req
prompt = no

[req_distinguished_name]
CN = $IP

[v3_req]
keyUsage = keyEncipherment, dataEncipherment
extendedKeyUsage = serverAuth
subjectAltName = IP:$IP
EOF

echo " Generating CA..."
openssl genrsa -out ca.key 2048
openssl req -x509 -new -nodes -key ca.key -subj "/CN=MQTT CA" -days $DAYS -out ca.crt

echo " Generating server key and CSR with SAN for IP: $IP..."
openssl genrsa -out server.key 2048
openssl req -new -key server.key -out server.csr -config $SAN_CONFIG

echo " Signing server certificate with CA..."
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial \
    -out server.crt -days $DAYS -sha256 -extfile $SAN_CONFIG -extensions v3_req

echo "Cleaning up..."
rm -f server.csr $SAN_CONFIG

echo " TLS certificates generated:"
ls -l ca.crt server.crt server.key

```

Fig 29 Bash Script to TLS

9. RESULTS

This section presents the outcomes of security evaluations conducted on the **IoTBot Security** system under various configurations. The results highlight the system's ability to detect, mitigate, and respond to multiple cyber threats—including SQL injection, DDoS attacks, spoofing, and unauthorized device access. Scenarios were tested both with and without MQTT broker-level protection (authentication, access control, and TLS), allowing a clear assessment of the individual and combined contributions of the **IoTBot engine** and the **secure MQTT infrastructure**. The findings demonstrate that while the IoTBot independently mitigates logic-based attacks, its effectiveness is greatly enhanced when integrated with broker-level encryption and authentication features, providing a comprehensive and layered defense architecture for smart IoT environments.

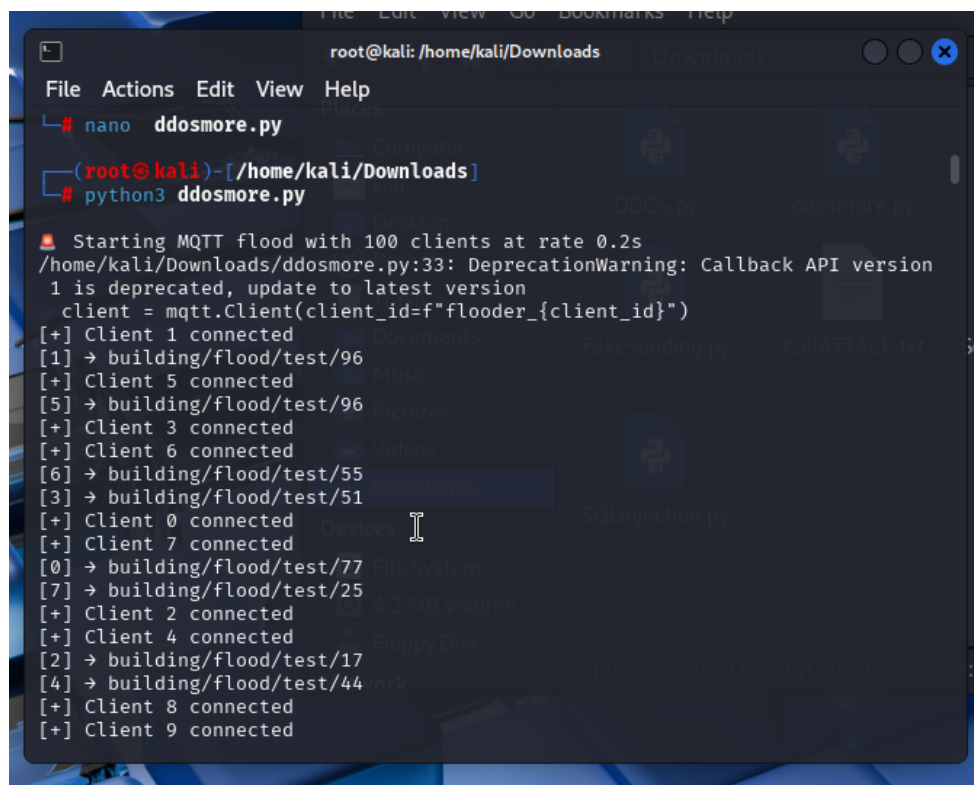
9.1 No Protection

When the system is deployed with **no security features enabled at the broker level** (i.e., no authentication, no access control, no TLS), the **IoTBot component alone is still capable of**

providing a baseline layer of protection. Independently of the MQTT broker, IoTBot continuously monitors every incoming MQTT message and applies its own anomaly detection logic. It successfully identifies and blocks **SQL injection attempts** by inspecting suspicious patterns in device IDs and defends against **DDoS attacks** using both global flood tracking and per-device rate limiting. This proves that even in an open and unsecured environment, the **IoTBot is effective at mitigating logical-level attacks** purely through payload analysis and behavioral heuristics.

8.1.1 DDOS

To evaluate the effectiveness of the IoTBot's DDoS mitigation logic, a simulated **MQTT flood attack** was launched using a custom script (ddosmore.py) from a Kali Linux environment. As shown in **Figure 30**, the script generated **100 concurrent MQTT clients**, each publishing messages to randomized topics at a high rate (0.2s intervals). This flood not only emulated excessive traffic but also introduced **a wave of unknown device IDs**, representing a real-world scenario of a coordinated distributed attack.

A screenshot of a terminal window titled 'root@kali: /home/kali/Downloads'. The window shows the execution of a script named 'ddosmore.py'. The prompt is '(root@kali)-[/home/kali/Downloads]'. The user enters '# python3 ddosmore.py'. The output shows the script starting an MQTT flood with 100 clients at a rate of 0.2s. It includes a deprecation warning for the Callback API version 1. The output lists the connection of several clients (0-9) to various topics like 'building/flood/test/96', 'building/flood/test/55', etc. The terminal has a dark background with light-colored text.

```
root@kali: /home/kali/Downloads
File Actions Edit View Help
# nano ddosmore.py
(root@kali)-[/home/kali/Downloads]
# python3 ddosmore.py

Starting MQTT flood with 100 clients at rate 0.2s
/home/kali/Downloads/ddosmore.py:33: DeprecationWarning: Callback API version
1 is deprecated, update to latest version
  client = mqtt.Client(client_id=f"flooder_{client_id}")
[+] Client 1 connected
[1] → building/flood/test/96
[+] Client 5 connected
[5] → building/flood/test/96
[+] Client 3 connected
[+] Client 6 connected
[6] → building/flood/test/55
[3] → building/flood/test/51
[+] Client 0 connected
[+] Client 7 connected
[0] → building/flood/test/77
[7] → building/flood/test/25
[+] Client 2 connected
[+] Client 4 connected
[2] → building/flood/test/17
[4] → building/flood/test/44
[+] Client 8 connected
[+] Client 9 connected
```

Fig 30 DDOS attack

Despite the MQTT broker having **no built-in rate limiting or TLS** active during this test, the IoTBot detected and responded autonomously. Using its layered detection logic (is_global_ddos(), is_ddos_from_new_ids(), and RATE_LIMIT), the system identified the traffic spike and activated **GLOBAL_DDOS_MODE**. Devices not previously registered in the system were automatically **quarantined** and their messages dropped.

As confirmed in **Figure 31**, the IoTBot successfully logged each anomaly in the PostgreSQL database. The **anomaly logs** table recorded thousands of entries tagged as "new_device_ddos"

events, all originating from fake or auto-generated device IDs such as fake_9000, fake_5429, and fake_1282. This evidences the system's ability to dynamically isolate malicious sources and throttle traffic in real time—**without broker-level protection**.

id [PK] integer	device_id text	payload jsonb
2243	fake_9000	{"event": "new_device_ddos", "payload": {"value": 2634.738468807901, "sensor": "fake_9000", "timestamp": "03/Jun/25 14:30:35"}}
2244	fake_5400	{"event": "new_device_ddos", "payload": {"value": 1627.8039510566223, "sensor": "fake_5400", "timestamp": "03/Jun/25 14:30:35"}}
2245	fake_5429	{"event": "new_device_ddos", "payload": {"value": 1963.4378162882838, "sensor": "fake_5429", "timestamp": "03/Jun/25 14:30:35"}}
2246	fake_2484	{"event": "new_device_ddos", "payload": {"value": 9348.022372528172, "sensor": "fake_2484", "timestamp": "03/Jun/25 14:30:35"}}
2247	fake_8490	{"event": "new_device_ddos", "payload": {"value": 3348.0969801954498, "sensor": "fake_8490", "timestamp": "03/Jun/25 14:30:35"}}
2248	fake_6698	{"event": "new_device_ddos", "payload": {"value": 5947.460862942466, "sensor": "fake_6698", "timestamp": "03/Jun/25 14:30:35"}}
2249	fake_1282	{"event": "new_device_ddos", "payload": {"value": 3181.1704885893864, "sensor": "fake_1282", "timestamp": "03/Jun/25 14:30:35"}}
2250	fake_1313	{"event": "new_device_ddos", "payload": {"value": 9527.833798449323, "sensor": "fake_1313", "timestamp": "03/Jun/25 14:30:35"}}
2251	fake_3008	{"event": "new_device_ddos", "payload": {"value": 3654.0430028490678, "sensor": "fake_3008", "timestamp": "03/Jun/25 14:30:35"}}
2252	fake_3332	{"event": "new_device_ddos", "payload": {"value": 1453.7465094880997, "sensor": "fake_3332", "timestamp": "03/Jun/25 14:30:35"}}
2253	fake_7499	{"event": "new_device_ddos", "payload": {"value": 9093.054103567763, "sensor": "fake_7499", "timestamp": "03/Jun/25 14:30:35"}}
2254	fake_8061	{"event": "new_device_ddos", "payload": {"value": 4031.729748189771, "sensor": "fake_8061", "timestamp": "03/Jun/25 14:30:35"}}

total rows: 7598 Query complete 00:00:00.113 LF Ln 2, Col 27

Fig 30 IoTSecBoT Mitigation DDOS

This test demonstrates that the **IoTBot can independently identify and mitigate high frequency distributed MQTT attacks**, offering an essential defense layer for any IoT network. However, for enhanced resilience, combining IoTBot with broker-level TLS, ACLs, and authentication is recommended to defend against more advanced threats like MITM or internal credential misuse.

8.1.2 SQL Injection

To evaluate the system's resilience against SQL injection attacks, a specially crafted MQTT payload was sent from a malicious Python script (SQLInjectionFREE.py). As shown in Figure 31, the attacker attempted to inject the following string into the device_id field: **sensor'; DROP TABLE users; --**

```
payload = {
    "device_id": "sensor'; DROP TABLE users; --",
    "sensor": "temperature",
    "value": 999,
    "timestamp": "01/Jun/25 12:00:00"
}

client.connect(BROKER, PORT)
client.loop_start()
client.publish(TOPIC, json.dumps(payload))
print("[!] SQLi payload sent.")
client.loop_stop()
```

Fig 31 SQL injection Attack

This payload mimics a classic SQL injection pattern designed to terminate a valid SQL query and execute a destructive command (in this case, DROP TABLE users). The message was sent without HMAC, bypassing authentication mechanisms and directly targeting the system's backend via the MQTT broker on port 1883 (no TLS, no auth).

Despite the broker allowing this message to pass, the **IoTBot security engine intercepted it**, evaluated the structure of the payload, and **identified the malicious input** using its `is_sql_like()` filter. As shown in **Figure 32**, the IoTBot flagged and **quarantined the sender**, logging the attempt in the `anomaly_logs` table with the event "sql_injection". The payload was **not executed**, and the targeted table remained intact.

	id [PK] integer	device_id text	payload jsonb
1	24	sensor; DROP TABLE users; --	{ "event": "quarantined", "payload": { "value": 999, "sensor": "temperature", "device_id": "sensor; DROP TABLE users; --" } }
2	23	sensor; DROP TABLE users; --	{ "event": "quarantined", "payload": { "value": 999, "sensor": "temperature", "device_id": "sensor; DROP TABLE users; --" } }

Fig 32 IoTSecBoT mitigating SQL injection

8.1.2 Spoofing

To validate the system's vulnerability in the absence of broker-level security, a controlled spoofing attack was executed from a Kali Linux machine. As shown in Figure 23, an attacker used the `mosquitto_pub` command to publish multiple fake MQTT messages to the topic `building/zone2/temperature/room1`. These messages were crafted to impersonate the legitimate device `sensor_temp_01` and carried a manually forged value of 28.5°C with the backdated timestamp "2025-06-03 15:45:00". The broker was running in open mode (port 1883), with no authentication or TLS enabled, allowing any client to connect and publish without restriction.

```

28.5, timestamp : 2025-06-03 15:45:00 }

(root@kali)-[~]
# mosquitto_pub -h 192.168.10.1 -p 1883 -t building/zone2/temperature/room1 -m '{"device_id":"sensor_temp_01","value":28.5,"timestamp":"2025-06-03 15:45:00"}'

(root@kali)-[~]
# mosquitto_pub -h 192.168.10.1 -p 1883 -t building/zone2/temperature/room1 -m '{"device_id":"sensor_temp_01","value":28.5,"timestamp":"2025-06-03 15:45:00"}'

(root@kali)-[~]
# mosquitto_pub -h 192.168.10.1 -p 1883 -t building/zone2/temperature/room1 -m '{"device_id":"sensor_temp_01","value":28.5,"timestamp":"2025-06-03 15:45:00"}'

(root@kali)-[~]
#
  
```

Fig 33 Spoofing Success

As observed in Figure 24, the spoofed message was successfully logged in the `sensor_data` table of the PostgreSQL database. It appears seamlessly alongside genuine data from real sensors, but the timestamp betrays the attack—standing out clearly as an outlier when compared to surrounding entries. While legitimate entries show timestamps clustered around "2025-06-03 14:17:xx", the spoofed value lands abruptly at "15:45:00", well outside the natural data flow. This provides hard, visual evidence that unauthenticated and unencrypted MQTT traffic can be spoofed without detection.

Data Output

Messages

Notifications

<

Fig 34 Spoofing with no HMAC

This experiment highlights a critical gap in security: although the IoTBot can detect malformed payloads and strange behavior, it cannot identify a well-formed, properly structured message that mimics a trusted device ID. Spoofed messages like these bypasses all application-level defenses unless the broker enforces:

- **Authentication (so only registered clients can publish)**
- **Access Control Lists (ACLs) (to limit topic access per user)**
- **TLS encryption (to prevent interception or manipulation in transit)**

The message shown in Figures 23 and 24 not only demonstrates the feasibility of spoofing but also underscores the non-negotiable need for broker-level protections. Only through the combined power of IoTBot monitoring and secure MQTT configurations can such attacks be reliably prevented.

8.1.2 MITM

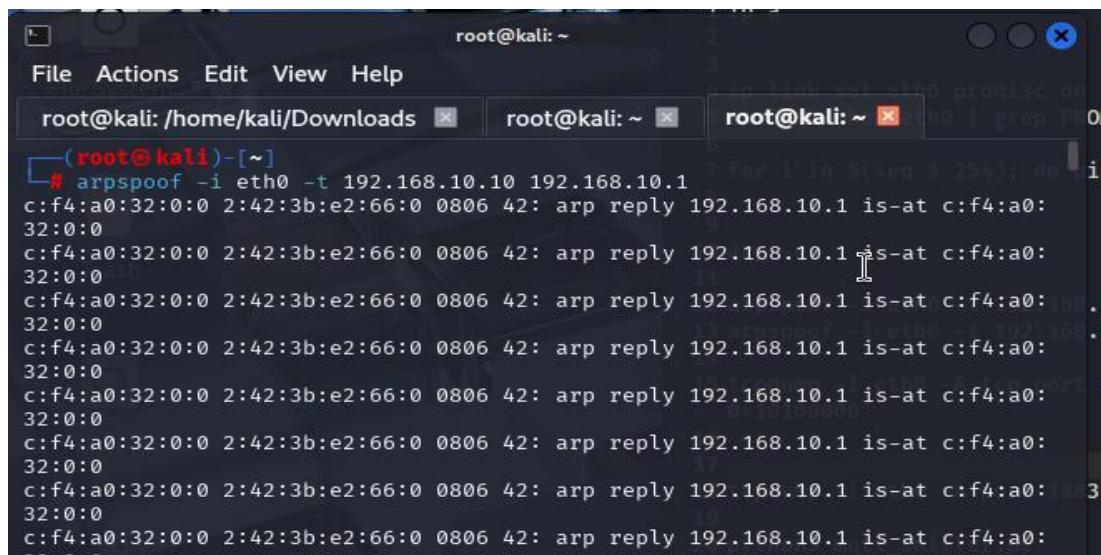
To demonstrate the system's vulnerability in the absence of TLS encryption or network-level defense, a full Man-in-the-Middle (MITM) attack was performed using a Kali Linux machine. As shown in Figures 25–31, the attacker first enabled promiscuous mode on their network

interface and executed a host discovery sweep using a scripted ping sweep (for i in \$(seq 1 254)...). This exposed all active devices and their IP-MAC bindings via the arp -n command.

```
root@kali: /home/kali/Downloads
File Actions Edit View Help
root@kali: /home/kali/Downloads x root@kali: ~ x root@kali: ~ x
(root@kali)-[~]
# arp -n
Address HWtype HWaddress Flags Mask If
192.168.10.10 ether 02:42:3b:e2:66:00 C et
192.168.10.22 ether 02:42:4d:84:83:00 C et
192.168.10.11 ether 02:42:39:d3:ed:00 C et
192.168.10.5 ether 02:42:c8:14:df:00 C et
192.168.122.1 ether 52:54:00:56:f2:8e C et
192.168.10.12 ether 02:42:68:73:b5:00 C et
192.168.10.1 ether 02:42:ba:e8:72:00 C et
192.168.10.2 ether 02:42:3d:b9:1f:00 C et
192.168.10.20 ether 02:42:de:6f:a3:00 C et
192.168.10.3 ether 02:42:f0:24:e7:00 C et
```

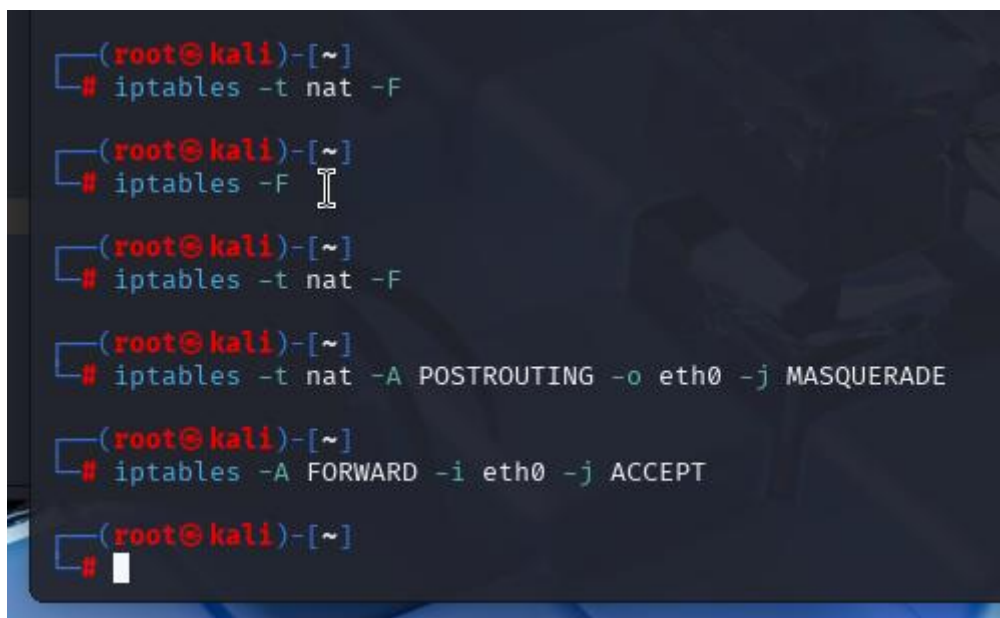
Fig 35. Attacker MITM ARP table.

Next, using arpspoof, the attacker launched a classic ARP poisoning attack to redirect traffic between the broker and a sensor device—masquerading as both sides simultaneously. Once this routing manipulation was in place, IP forwarding and NAT masquerading were enabled using echo 1 > /proc/sys/net/ipv4/ip_forward and iptables, allowing traffic to transparently flow through the attacker’s machine without interruption to the devices.

A terminal window on a Kali Linux machine. The prompt is root@kali: ~. The command arpspoof -i eth0 -t 192.168.10.10 192.168.10.1 has been executed. The output shows a series of ARP replies being sent from the Kali machine to the target IP 192.168.10.1. Each line shows the source MAC address (c:f4:a0:32:0:0), the destination MAC address (2:42:3b:e2:66:0), the protocol (0806), the length (42), the type (arp reply), the destination IP (192.168.10.1), and the source IP (192.168.10.1).

```
root@kali: ~  
File Actions Edit View Help  
root@kali: /home/kali/Downloads x root@kali: ~ x root@kali: ~ x  
(root@kali)-[~]  
# arpspoof -i eth0 -t 192.168.10.10 192.168.10.1  
c:f4:a0:32:0:0 2:42:3b:e2:66:0 0806 42: arp reply 192.168.10.1 is-at c:f4:a0:  
32:0:0  
c:f4:a0:32:0:0 2:42:3b:e2:66:0 0806 42: arp reply 192.168.10.1 is-at c:f4:a0:  
32:0:0  
c:f4:a0:32:0:0 2:42:3b:e2:66:0 0806 42: arp reply 192.168.10.1 is-at c:f4:a0:  
32:0:0  
c:f4:a0:32:0:0 2:42:3b:e2:66:0 0806 42: arp reply 192.168.10.1 is-at c:f4:a0:  
32:0:0  
c:f4:a0:32:0:0 2:42:3b:e2:66:0 0806 42: arp reply 192.168.10.1 is-at c:f4:a0:  
32:0:0  
c:f4:a0:32:0:0 2:42:3b:e2:66:0 0806 42: arp reply 192.168.10.1 is-at c:f4:a0:  
32:0:0  
c:f4:a0:32:0:0 2:42:3b:e2:66:0 0806 42: arp reply 192.168.10.1 is-at c:f4:a0:  
32:0:0  
c:f4:a0:32:0:0 2:42:3b:e2:66:0 0806 42: arp reply 192.168.10.1 is-at c:f4:a0:  
32:0:0  
c:f4:a0:32:0:0 2:42:3b:e2:66:0 0806 42: arp reply 192.168.10.1 is-at c:f4:a0:  
32:0:0
```

Fig 36. MITM Spoofing the sensor

A terminal window on a Kali Linux machine showing the configuration of iptables rules. The rules are: 1. iptables -t nat -F (flush nat table), 2. iptables -F (flush all tables), 3. iptables -t nat -F (flush nat table), 4. iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE (enable masquerading on eth0), 5. iptables -A FORWARD -i eth0 -j ACCEPT (allow forwarding on eth0).

```
(root@kali)-[~]  
# iptables -t nat -F  
(root@kali)-[~]  
# iptables -F  
(root@kali)-[~]  
# iptables -t nat -F  
(root@kali)-[~]  
# iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE  
(root@kali)-[~]  
# iptables -A FORWARD -i eth0 -j ACCEPT  
(root@kali)-[~]  
#
```

Fig 37. Rules to send back the traffic to the original victim

Finally, as shown in Figure 32 (Sniffer Output), a custom Sniffer.py script was run to monitor MQTT port 1883 on interface eth0. Without TLS enabled, MQTT credentials were captured in plaintext, exposing sensitive authentication data:

```
valid_lft 3516sec preferred_lft 3516sec
inet6 fe80::9b7c:4d61:e655:dcfd/64 scope link noprefixroute
valid_lft forever preferred_lft forever

4 (root@kali)-[~]
# cd /home/kali/Downloads

6 (root@kali)-[/home/kali/Downloads]
# ls
KaliATTACK.txt Sniffer.py

8 (root@kali)-[/home/kali/Downloads]
# python3 Sniffer.py
Sniffing on eth0 - logs → /share/creds.txt
2025-06-03T15:43:03 CID= USER=sensor_user PASS=sensorpass
2025-06-03T15:43:03 CID= USER=sensor_user PASS=sensorpass
```

Fig 38.MITM Sniffing Success

9.2 Protection

As it is shown in the section **No Protection**, **IoTBot alone cannot defend against certain network-level attacks**, such as **Man-in-the-Middle (MitM)** or **spoofing**, where malicious actors forge packet origins or intercept messages during transit. These types of threats **require cryptographic safeguards**. That's why **authentication and HMAC-based message integrity checks** are essential: they ensure only known devices can publish data and that each message's content has not been tampered with. For even stronger protection—especially against MitM—**TLS encryption** must be enabled to secure MQTT sessions end-to-end. In summary, while IoTBot acts as a powerful last line of defense against logic-based attacks, **a truly secure system requires cooperation between application-level detection (IoTBot) and transport-level protection (MQTT + TLS + Auth)**

8.2.1 Spoofing

In a second spoofing test, a more secure scenario was evaluated: the MQTT system remained in open mode (no authentication or TLS), but HMAC message integrity checking was enabled at the application level through IoTBot. As shown in Figure 35, the attacker attempted to manually spoof a message using `mosquitto_pub`, impersonating `sensor_temp_01` and publishing a temperature value of 28.5°C with a forged timestamp.

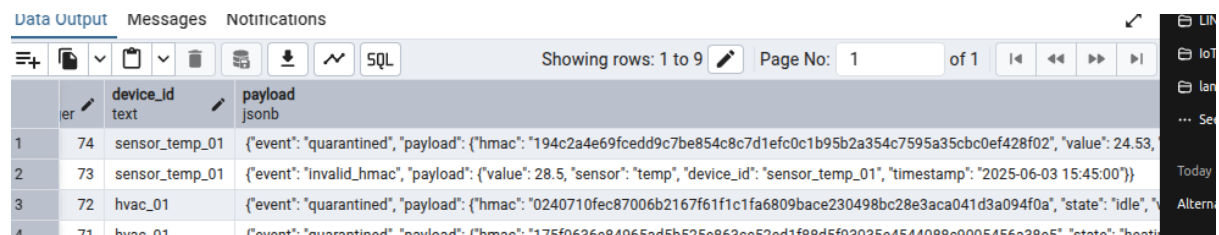
```
(root@kali)-[/home/kali/Downloads]
# mosquitto_pub -h 192.168.10.1 -p 1883 -t building/zone2/temperature/room1
-m '{"device_id":"sensor_temp_01","sensor":"temp","value":28.5,"timestamp":"
2025-06-03 15:45:00"}'

(root@kali)-[/home/kali/Downloads]
#
```

Fig 39 Spoofing Failure due to HMAC

However, unlike in the earlier spoofing test, this message lacked a valid HMAC signature, which is mandatory when `ENABLE_HMAC=true`. As shown in Figure 36, the IoTBot identified the message as forged and took two immediate actions:

1. Logged the event with the label "invalid_hmac", indicating that the payload failed cryptographic verification.
2. Quarantined the sender (sensor_temp_01) to block future traffic and prevent replay or flooding attempts from the same identity.



	device_id	payload
1	74 sensor_temp_01	{\"event\": \"quarantined\", \"payload\": {\"hmac\": \"194c2a4e69fcedd9c7be854c8c7d1efc0c1b95b2a354c7595a35cbc0ef428f02\", \"value\": 24.53, \"timestamp\": \"2025-06-03 15:45:00\"}}
2	73 sensor_temp_01	{\"event\": \"invalid_hmac\", \"payload\": {\"value\": 28.5, \"sensor\": \"temp\", \"device_id\": \"sensor_temp_01\", \"timestamp\": \"2025-06-03 15:45:00\"}}
3	72 hvac_01	{\"event\": \"quarantined\", \"payload\": {\"hmac\": \"0240710fec87006b2167f61f1c1fa6809bace230498bc28e3aca041d3a094f0a\", \"state\": \"idle\", \"timestamp\": \"2025-06-03 15:45:00\"}}
4	71 hvac_01	{\"event\": \"quarantined\", \"payload\": {\"hmac\": \"175f0636e8d965ad5b575e863ce57ed1f88d5f93035e454d088e9005456a38e5\", \"state\": \"heating\", \"timestamp\": \"2025-06-03 15:45:00\"}}

Fig 40 IoTSecBot mitigation Spoofing

This result demonstrates that HMAC-based integrity enforcement alone is sufficient to detect and block spoofed messages, even when sent over unsecured MQTT channels (port **1883**, no TLS). The attacker had no access to the shared secret (HMAC_SECRET) used to generate valid hashes, making it impossible to forge a legitimate signature.

The success of this defense illustrates that even without encryption or authentication at the transport level, HMAC offers strong protection against message forgery. It ensures that only devices with knowledge of the secret key can produce accepted data. However, when combined with TLS and broker authentication, HMAC becomes part of a robust, multi-layered defense system that can withstand both payload-level and transport-level attacks.

8.2.2 MITM

To evaluate the effectiveness of **TLS encryption** in protecting against passive MITM attacks, a sniffing attempt was conducted using both `tcpdump` and **Wireshark** while TLS was enabled on the MQTT broker (port **8883**). As shown in **Figure 41**, the attacker first used `tcpdump` to capture packets on interface `eth0`, filtering traffic on port 8883. Although the attacker could see connections and data flowing between clients and the broker, the payload content was fully encrypted and unreadable.


```
(root@kali)-[/home/kali/Downloads]
# python3 Sniffer.py
Sniffing on eth0 - logs → /share/creds.txt

1883:1 building/zone2/temperature
room1:m
{"device_id":"sensor_temp_01","sens
on temp value":28.5,"timestamp":

10:55:53.538827 IP 192.168.10.10.34321 > 192.168.10.1.8883: Flags [P.], seq 1808807851:1808808093, ack 2248320389, win 501,
options [nop,nop,TS val 3564054686 ecr 3473051133], length 242
10:55:53.538865 IP 192.168.10.100.34321 > 192.168.10.1.8883: Flags [P.], seq 1808807851:1808808093, ack 2248320389, win 501,
options [nop,nop,TS val 3564054686 ecr 3473051133], length 242
10:55:53.539794 IP 192.168.10.1.8883 > 192.168.10.100.34321: Flags [.], ack 242, win 501, options [nop,nop,TS val 3473056032
ecr 3564054686], length 0
10:55:53.539820 IP 192.168.10.1.8883 > 192.168.10.10.34321: Flags [.], ack 242, win 501, options [nop,nop,TS val 3473056032
ecr 3564054686], length 0
10:55:53.642721 IP 192.168.10.1.8883 > 192.168.10.22.54831: Flags [P.], seq 2301996967:2301997161, ack 1614765817, win 501,
options [nop,nop,TS val 3857818534 ecr 62851102], length 194
10:55:53.642722 IP 192.168.10.1.8883 > 192.168.10.100.34321: Flags [P.], seq 1:195, ack 242, win 501, options [nop,nop,TS val
3473056135 ecr 3564054686], length 194
10:55:53.642767 IP 192.168.10.1.8883 > 192.168.10.10.34321: Flags [P.], seq 1:195, ack 242, win 501, options [nop,nop,TS val
3473056135 ecr 3564054686], length 194
10:55:53.643254 IP 192.168.10.10.34321 > 192.168.10.1.8883: Flags [.], ack 195, win 501, options [nop,nop,TS val 3564054791
ecr 3473056135], length 0
10:55:53.643270 IP 192.168.10.100.34321 > 192.168.10.1.8883: Flags [.], ack 195, win 501, options [nop,nop,TS val 3564054791
ecr 3473056135], length 0
10:55:55.613579 IP 192.168.10.1.8883 > 192.168.10.22.54831: Flags [P.], seq 194:417, ack 1, win 501, options [nop,nop,TS val
3857820505 ecr 62855137], length 223
10:55:58.647651 IP 192.168.10.1.8883 > 192.168.10.22.54831: Flags [P.], seq 417:611, ack 1, win 501, options [nop,nop,TS val
3857823539 ecr 62857108], length 194
10:55:58.647653 IP 192.168.10.1.8883 > 192.168.10.100.34321: Flags [P.], seq 195:389, ack 242, win 501, options [nop,nop,TS
val 3473061140 ecr 3564054791], length 194
10:55:58.647754 IP 192.168.10.1.8883 > 192.168.10.10.34321: Flags [P.], seq 195:389, ack 242, win 501, options [nop,nop,TS v
al 3473061140 ecr 3564054791], length 194
10:55:58.854230 IP 192.168.10.1.8883 > 192.168.10.100.34321: Flags [P.], seq 195:389, ack 242, win 501, options [nop,nop,TS
val 3473061347 ecr 3564054791], length 194
```

Fig 41 Tcpcdump to Sniff port 8883

This result is confirmed in **Figure 42**, captured using Wireshark. The screenshot shows a TLSv1.2 Application Data packet being intercepted, but the payload—highlighted in hexadecimal—is completely obfuscated. No usernames, passwords, sensor values, device IDs, or MQTT topic names are visible. Unlike earlier tests where the attacker extracted clear-text credentials (sensor_user : sensorpass) via sniffing, **this time the traffic content remained protected end-to-end.**

The critical difference lies in **TLS's encryption of the application layer**, ensuring:

- **Confidentiality:** even if a MITM is in place, the attacker cannot decipher message content
- **Integrity:** any tampering would break the TLS session
- **Authentication** (optional but supported): via broker-side certificates and optional client certs

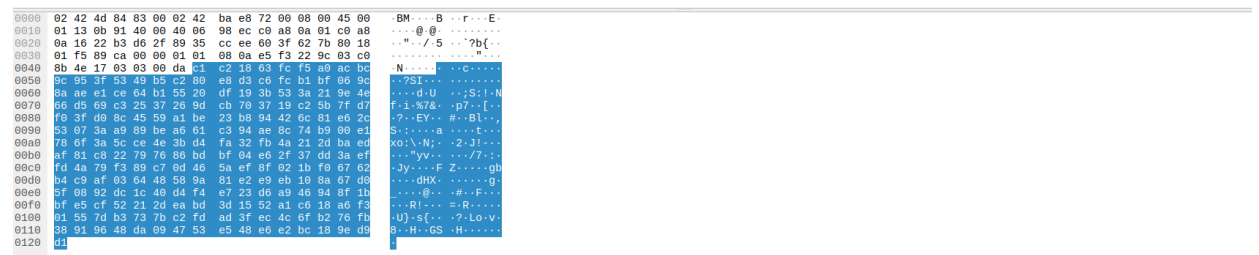
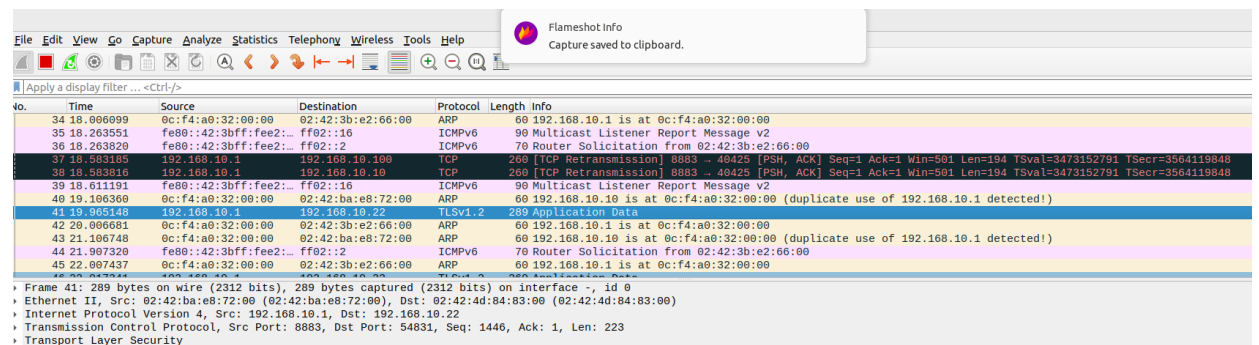


Fig 42 WireShark Link Kali Linux Switch

This experiment proves that **TLS completely neutralizes sniffing-based MITM attacks**. Even with successful ARP spoofing and full traffic redirection, the attacker **gains no useful information**. Therefore, TLS must be considered **mandatory** for any IoT deployment transmitting sensitive data over MQTT.

10. CONCLUSION

- The project demonstrated that relying on a single layer of security is insufficient for protecting MQTT-based IoT systems. While IoTBot can independently detect and mitigate logical threats like DDoS, SQL injection, and malformed payloads, it is only when combined with broker-level protections—TLS encryption, authentication, and access control—that the system becomes resilient against advanced threats such as spoofing and MITM attacks. A layered approach ensures complete coverage across the application, transport, and identity layers.
- Through live attack scenarios—including DDoS floods, spoofed sensor injections, credential sniffing, and ARP-based MITM—the system's defenses were rigorously tested under real-world conditions. These tests confirmed that HMAC protected against spoofing, TLS rendered MITM attacks ineffective, and IoTBot mitigated SQL injection and flooding in open modes. The results underscore the importance of building both proactive detection mechanisms and reactive response logic into modern IoT systems.

